

Microservices

Service Oriented Architecture 2.0

Gesamtinhaltsverzeichnis

1	Einführung	1-3
1.1	Von Monolithen und Services	1-3
1.1.1	Eine Anwendung.....	1-3
1.1.2	Monolith	1-3
1.1.3	Was wäre "besser"?.....	1-3
1.1.4	Gekapselte Services.....	1-4
1.1.5	Das API.....	1-4
1.1.6	Dokumentation ist alles.....	1-4
1.1.7	Realisierung der API-Beschreibung	1-5
1.1.8	Stereotypen zur Vermeidung von Redundanzen	1-5
1.1.9	Definition der Service-Kommunikation	1-6
1.2	Was sind Microservices	1-7
1.2.1	Problemstellung	1-7
1.2.2	Architektur mit gemeinsamen Ressourcen	1-7
1.2.3	Definitionen.....	1-8
1.2.4	Microservices mit gekapselten Ressourcen.....	1-8
1.2.5	Microservices kompakt	1-8
1.2.6	Warum "Micro"?	1-9
1.2.7	Microservices und Daten-Redundanzen	1-9
1.2.8	Das CAP-Theorem.....	1-9
1.2.9	Keine gemeinsame Schnittmenge!	1-10
1.2.10	BASE: Ein Schlupfloch.....	1-10
1.2.11	ACID und BASE.....	1-11
1.2.12	Synchronisation durch Implementierung.....	1-12
1.2.13	Synchronisation durch Data Pump.....	1-13
1.2.14	Synchronisation durch Data Service	1-13
1.3	Modellierung	1-14
1.3.1	Web-Services mit SOAP – Eine Lösung?	1-14
1.3.2	Benutzung von SOAP-basierten Web-Services	1-14
1.3.3	Beispiel eines SOAP-Web Services.....	1-15
1.3.4	Das Web interessiert sich nicht für SOAP!.....	1-15
1.3.5	Der Representational State Transfer	1-16
1.3.6	Kern-Konzepte	1-16
1.3.7	Das http-Protokoll	1-17

1.3.8	Elemente der http-Spezifikation	1-17
1.3.9	Elemente der http-Spezifikation II	1-18
1.3.10	MimeTypes	1-18
1.3.11	REST und http	1-18
1.3.12	Mapping REST – http.....	1-18
1.3.13	Konzeption eines RESTful Services: Neuanlage	1-19
1.3.14	Konzeption eines RESTful Services: Update.....	1-19
1.3.15	Konzeption eines RESTful Services: Delete	1-20
1.3.16	Service-Beschreibung	1-20
1.3.17	REST Services	1-21
1.3.18	Ein komplett anderer Ansatz: HATEOAS.....	1-21
1.3.19	Wie groß ist "Micro"	1-21
2	Software-Entwicklung.....	2-3
2.1	Java, JEE und Spring Boot.....	2-3
2.1.1	Java	2-3
2.1.2	JEE	2-3
2.1.3	Spring Source	2-3
2.1.4	Spring Boot.....	2-4
2.2	Build-Prozess	2-5
2.2.1	Kriterien für den Build-Prozess	2-5
2.2.2	Bestandteile	2-5
2.2.3	Beispiel: Apache Maven	2-5
2.2.4	Build-Prozess für Java-Projekte.....	2-6
2.2.5	Apache Maven.....	2-6
2.2.6	Maven-Konfiguration: Das interne Repository	2-7
2.2.7	Maven-Konfiguration: Ausbringen eigener Artefakte	2-7
2.2.8	Weitere Elemente der Parent-POM	2-8
2.2.9	Beispiel: Sonatype Nexus.....	2-8
2.2.10	Grundsätzliche Organisation.....	2-9
2.2.11	Konfiguration.....	2-9
2.2.12	Nexus Maven-Repository.....	2-10
2.3	Integration und Überwachung	2-11
2.3.1	Problemstellung	2-11
2.3.2	Jolokia.....	2-11
3	Container für Microservices.....	3-3
3.1	Applikationsserver	3-3
3.1.1	Allgemeines	3-3
3.1.2	Deployment.....	3-3
3.1.3	Überwachung.....	3-3

3.1.4	Der Applikationsserver.....	3-4
3.1.5	Inbound Connectors.....	3-4
3.1.6	Aufgaben des Inbound Connectors	3-5
3.1.7	JEE Komponenten	3-5
3.1.8	JEE Komponenten	3-5
3.1.9	Outbund Connectors.....	3-6
3.1.10	Beispiele für Outbound Connectors	3-6
3.2	Docker	3-7
3.2.1	Was ist ein Container?	3-7
3.2.2	Container-Isolation.....	3-7
3.2.3	Bestandteile	3-7
3.2.4	Docker und (Micro)Services.....	3-8
3.2.5	Microservices mit Docker-Containern	3-8
3.2.6	Starten eines ersten Containers	3-9
3.2.7	Docker-Workflow: 1. und 2. Lauf.....	3-10
3.2.8	Lebenszyklus des Containers	3-10
3.2.9	Docker im Gesamtbild.....	3-10
3.2.10	Images und Layers	3-11
3.2.11	Beispiel: Ein Java-Image (https://microbadger.com/images/java)	3-11
3.2.12	Container	3-12
3.2.13	Inter-Container-Kommunikation	3-12
3.2.14	Kommandoreferenz	3-13
3.2.15	Workflow	3-13
3.2.16	Dockerfile für eine Java-Umgebung.....	3-14
3.2.17	Dockerfile Referenz	3-14
3.2.18	Gekapselte Services mit Docker	3-15
4	Literatur und Quellen.....	4-3

1

Einführung

1.1	Von Monolithen und Services	1-3
1.1.1	Eine Anwendung.....	1-3
1.1.2	Monolith	1-3
1.1.3	Was wäre "besser"?.....	1-3
1.1.4	Gekapselte Services	1-4
1.1.5	Das API.....	1-4
1.1.6	Dokumentation ist alles	1-4
1.1.7	Realisierung der API-Beschreibung	1-5
1.1.8	Stereotypen zur Vermeidung von Redundanzen	1-5
1.1.9	Definition der Service-Kommunikation	1-6
1.2	Was sind Microservices	1-7
1.2.1	Problemstellung	1-7
1.2.2	Architektur mit gemeinsamen Ressourcen	1-7
1.2.3	Definitionen	1-8
1.2.4	Microservices mit gekapselten Ressourcen	1-8
1.2.5	Microservices kompakt	1-8
1.2.6	Warum "Micro"?	1-9
1.2.7	Microservices und Daten-Redundanzen	1-9
1.2.8	Das CAP-Theorem.....	1-9
1.2.9	Keine gemeinsame Schnittmenge!	1-10
1.2.10	BASE: Ein Schlupfloch.....	1-10

1.2.11	ACID und BASE.....	1-11
1.2.12	Synchronisation durch Implementierung.....	1-12
1.2.13	Synchronisation durch Data Pump	1-13
1.2.14	Synchronisation durch Data Service	1-13
1.3	Modellierung.....	1-14
1.3.1	Web Services mit SOAP – Eine Lösung?	1-14
1.3.2	Benutzung von SOAP-basierten Web Services	1-14
1.3.3	Beispiel eines SOAP-WebServices	1-15
1.3.4	Das Web interessiert sich nicht für SOAP!.....	1-15
1.3.5	Der Representational State Transfer	1-16
1.3.6	Kern-Konzepte.....	1-16
1.3.7	Das http-Protokoll	1-17
1.3.8	Elemente der http-Spezifikation	1-17
1.3.9	Elemente der http-Spezifikation II	1-18
1.3.10	MimeTypes	1-18
1.3.11	REST und http	1-18
1.3.12	Mapping REST – http.....	1-18
1.3.13	Konzeption eines RESTful Services: Neuanlage	1-19
1.3.14	Konzeption eines RESTful Services: Update.....	1-19
1.3.15	Konzeption eines RESTful Services: Delete	1-20
1.3.16	Service-Beschreibung.....	1-20
1.3.17	REST Services	1-21
1.3.18	Ein komplett anderer Ansatz: HATEOAS.....	1-21
1.3.19	Wie groß ist "Micro"	1-21

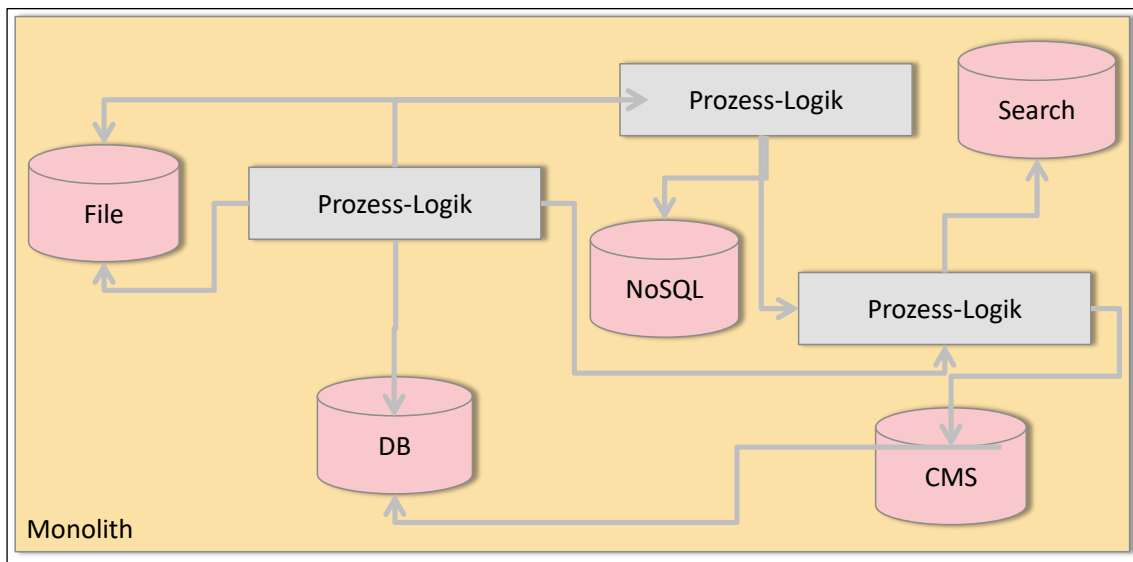
1 Einführung

1.1 Von Monolithen und Services

1.1.1 Eine Anwendung

- Primärfokus:
 - Anwendung muss fachlich funktionieren
 - Nicht-funktionale Aspekte wie Performance und Ausfallsicherheit
- Eher uninteressant sind:
 - Details der technischen Modellierung
 - Wartbarkeit und Erweiterbarkeit, Wiederverwendung

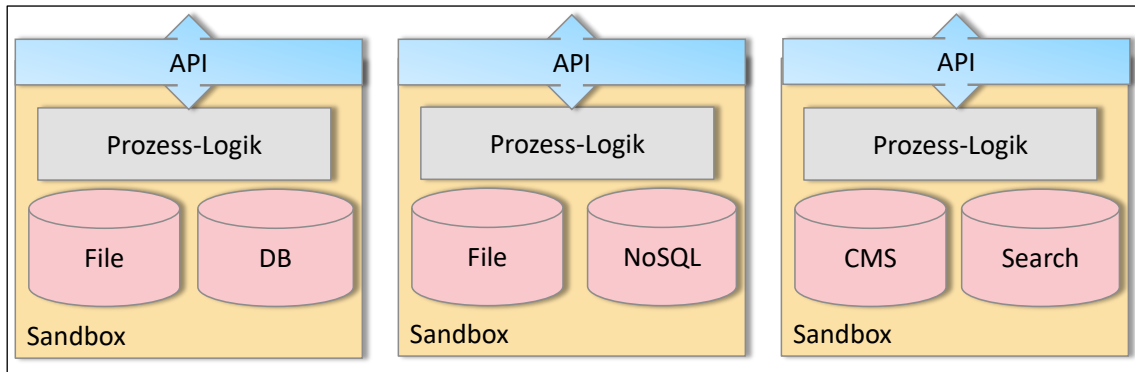
1.1.2 Monolith



1.1.3 Was wäre "besser"?

- Kapselung
 - Öffentliches „API“
 - Rein interne Implementierung
- Vermeidung von Redundanzen
 - Wartbarkeit
- Dependency Management
 - Statisch determinierte Abhängigkeiten oder
 - lose gekoppelte Systeme
 - Damit auch leichtere Erweiterbarkeit

1.1.4 Gekapselte Services



1.1.5 Das API

- Besteht abstrakt aus
 - Datentypen
 - Operationen
 - synchron und asynchron
- Konkretisierung mit weiteren Informationen
 - Authentifizierung
 - Aufruf über Netzwerk
 - Nicht-funktionale Teile
 - Antwortzeiten
 - Skalierbarkeit
 - Fehlertoleranz

1.1.6 Dokumentation ist alles

- Auch eine auf Microservices beruhende Architektur steht und fällt mit der Dokumentation
 - Vollständig
 - so natürlich utopisch
 - "eventual complete"
 - Aktuell
 - Unmissverständlich
 - Prägnant
 - Synchron zur Realisierung

1.1.7 Realisierung der API-Beschreibung

- (Formale) Prosa-Texte
 - Fachdokumentationen
- Unified Modelling Language
- Service-Beschreibung mit
 - Web Service Description Language
 - WSDL ist eine Spezifikation des W3W-Konsortiums
 - Primär nur in Kombination mit SOAP-basierten Web-Services
- Swagger
 - Proprietäre Beschreibungssprache
 - swagger.io
 - UI und Code Generatoren für verschiedene Sprachen werden angeboten
 - Primär ausgerichtet auf RESTful Web Services
- Elemente einer Programmiersprache
 - Beispielsweise Java-Interfaces und Annotationen

1.1.8 Stereotypen zur Vermeidung von Redundanzen

- Grundprinzipien der Software-Qualität gelten auch für die Dokumentation
 - Don't repeat yourself!
- UML-Stereotypen
 - Prägnanter Name
 - Gültigkeitsbereich
 - Auswirkung
 - Beispiel:
 - Ein <<stateless>> System baut kein Konversationsgedächtnis auf
- Stereotypen sind auch unter dem Namen Annotations bekannt

1.1.9 Definition der Service-Kommunikation

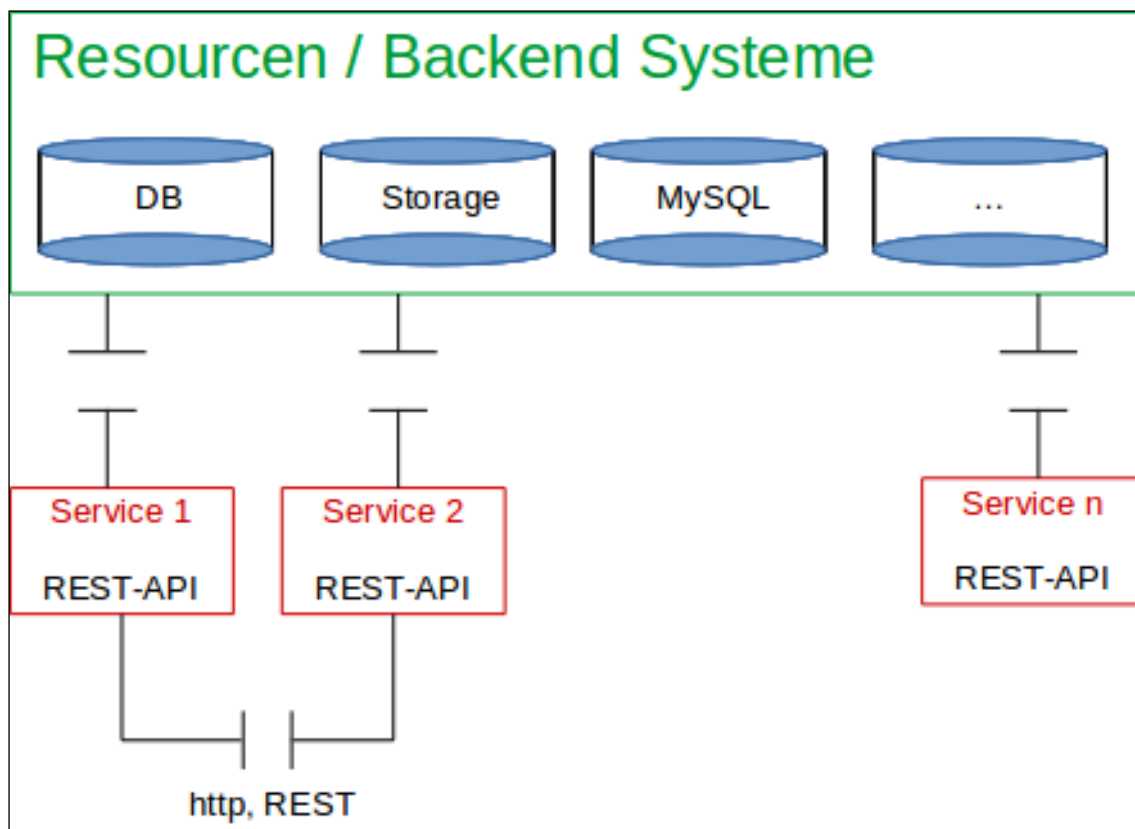
- Etablierte Protokolle stehen zur Verfügung
 - SOAP
 - REST
 - Messaging
 - Java RMI
- Was ist "richtig"?
 - Messaging-System sind bleibt
 - Trend geht zu REST
 - SOAP wird häufig als unnötig komplex gewertet
 - Java RMI ist eine Nischen-Lösung

1.2 Was sind Microservices

1.2.1 Problemstellung

- Eine vollständige API-Beschreibung für den Aufruf des Services ist leider nicht genügend!
- Services können durch die eigentlich "interne" und gekapselte Implementierung trotzdem gekoppelt bleiben
 - Gemeinsam genutzte Backend-Systeme!
- Damit hat jeder Service damit zwei APIs, die beschrieben und stabil gehalten werden müssen

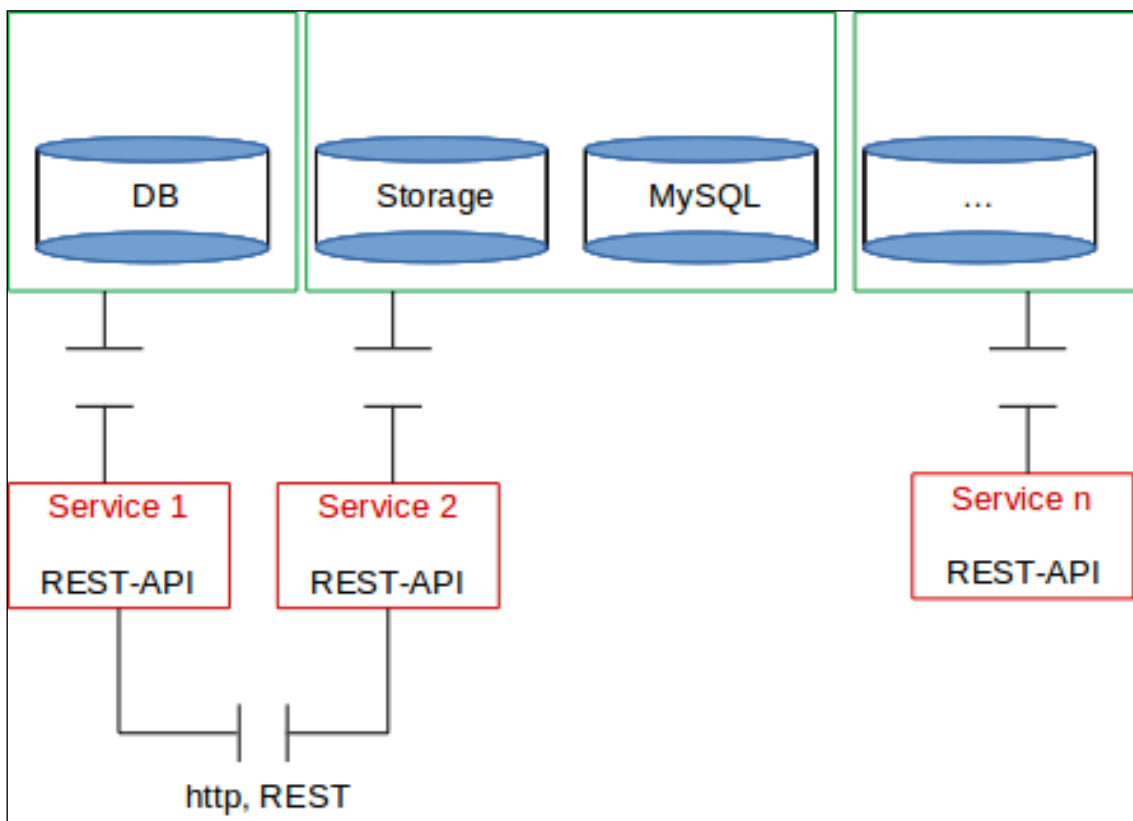
1.2.2 Architektur mit gemeinsamen Ressourcen



1.2.3 Definitionen

- WIKI
<https://de.wikipedia.org/wiki/Microservices>
- Martin Fowler
<https://martinfowler.com/articles/microservices.html>
- Microservice.io
<http://microservices.io/patterns/microservices.html>
 - Alles noch etwas im Fluss...

1.2.4 Microservices mit gekapselten Ressourcen



1.2.5 Microservices kompakt

- Der Microservice wird durch ein versioniertes API vollständig beschrieben
- Die Implementierung ist komplett gekapselt
 - Kriterium: Selbst eine Neuimplementierung ist möglich, ohne andere Services zu ändern
- Jeder Microservice bringt seine Backend-Ressourcen mit
 - Kriterium: Ein Austausch eines Backend-Systems ist möglich

1.2.6 Warum "Micro"?

- Ein Microservice
 - "besteht nur aus einer überschaubaren Anzahl von Programmzeilen"
 - "Kann in zwei Wochen neu entwickelt werden"
 - "Wird von kleinen Teams verantwortet"
- Eine irreführende Bezeichnung
 - Selbst ein "kleiner" Service kann die Anforderungen an Microservices verletzen
 - Auch eine komplexe (vielleicht intern sogar monolithisch realisierte!) Anwendung kann als Microservice betrieben werden

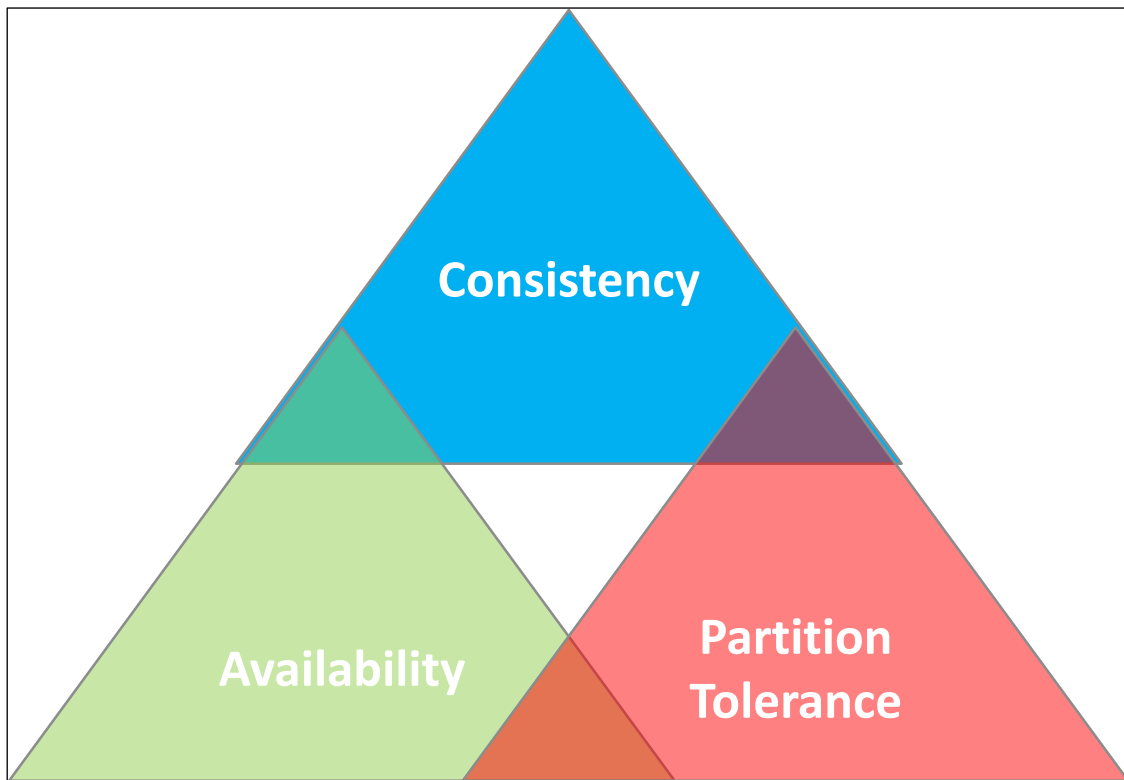
1.2.7 Microservices und Daten-Redundanzen

- Nachdem jeder Microservice seine Ressourcen intern kapselt, müssen häufig Daten-Redundanzen in Kauf genommen werden
- Häufig werden Redundanzen jedoch auch schon auf Grund von "Big&Fast Data" toleriert
- Damit haben Microservices einen direkten Bezug zu NoSQL-Themen!

1.2.8 Das CAP-Theorem

- Consistency
 - Alle Knoten haben jederzeit denselben Informationsstand
- Availability
 - Jeder Client bekommt garantiert Antwort
 - Solange zumindest ein Knoten im Cluster läuft
- Partition Tolerance
 - Das System toleriert
 - Den Ausfall von Knoten
 - Den Ausfall des Netzwerks zwischen Knoten

1.2.9 Keine gemeinsame Schnittmenge!



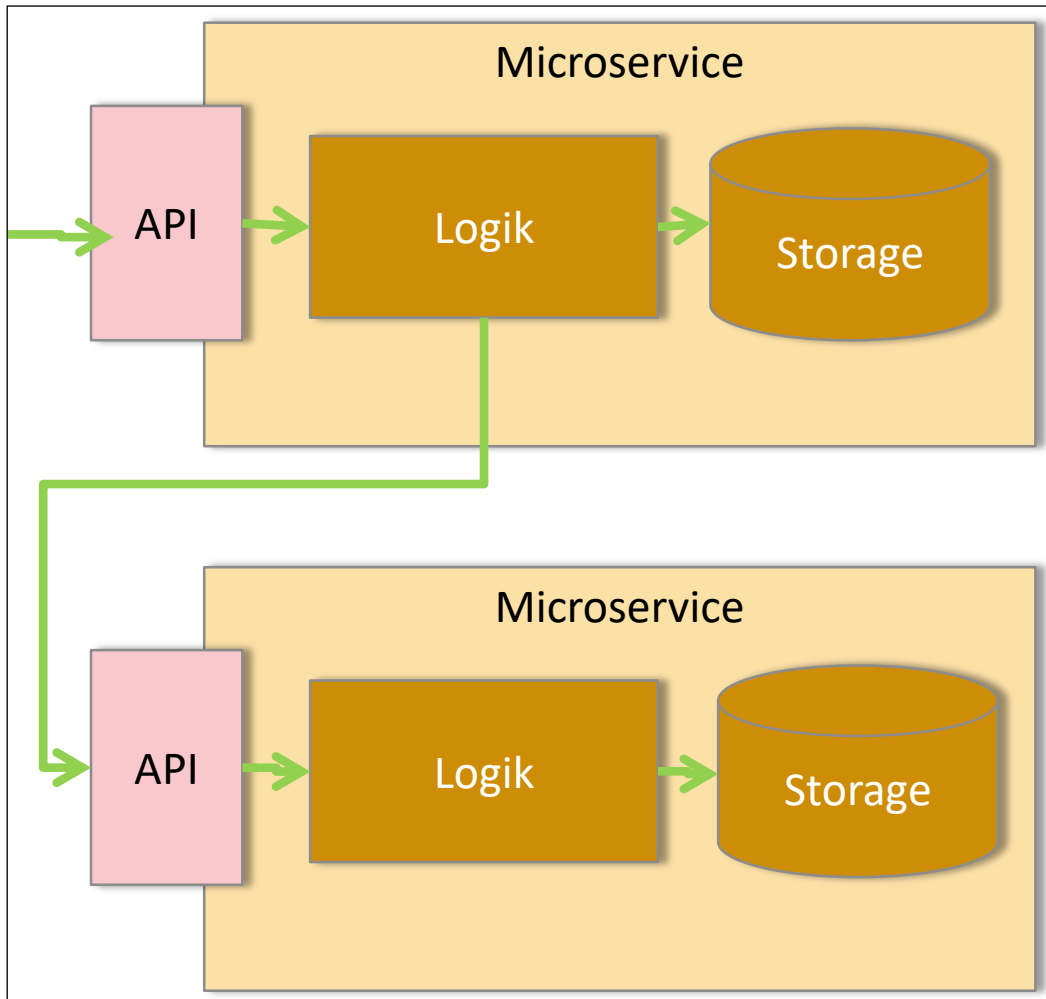
1.2.10 BASE: Ein Schlupfloch

- Consistency
 - Alle Knoten haben **jederzeit** denselben Informationsstand
- Eventually Consistent
 - Änderungen des Datenbestandes werden zeitlich versetzt an die anderen Knoten propagiert
 - Das System wird schlussendlich konsistent
- BASE
 - **B**asically **A**vailable
 - **S**oft state
 - **E**ventual consistency

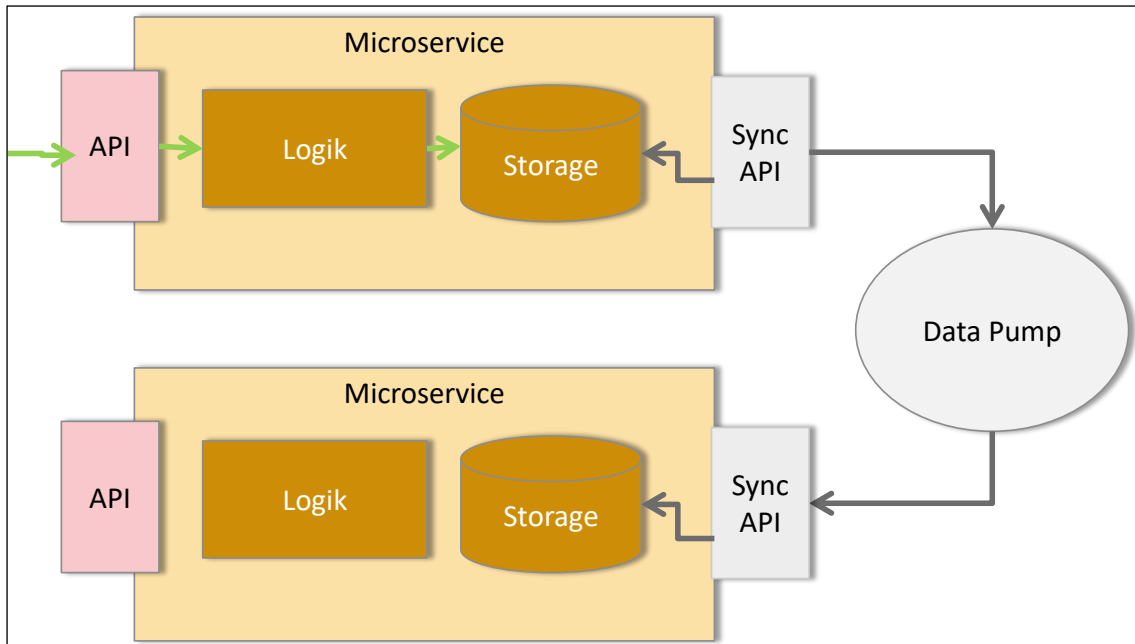
1.2.11 ACID und BASE

- ACID
 - Starke Konsistenz der Daten
 - Transaktionssteuerung
 - Transaction-Isolation
 - Bei Bedarf Zwei-Phasen-Commit
 - Relativ komplexe Entwicklung
- BASE
 - Schwache Konsistenz
 - Hohe Verfügbarkeit
 - Schnelligkeit
 - Bei Bedarf "Fire-and-forget"
 - Leichtere Entwicklung

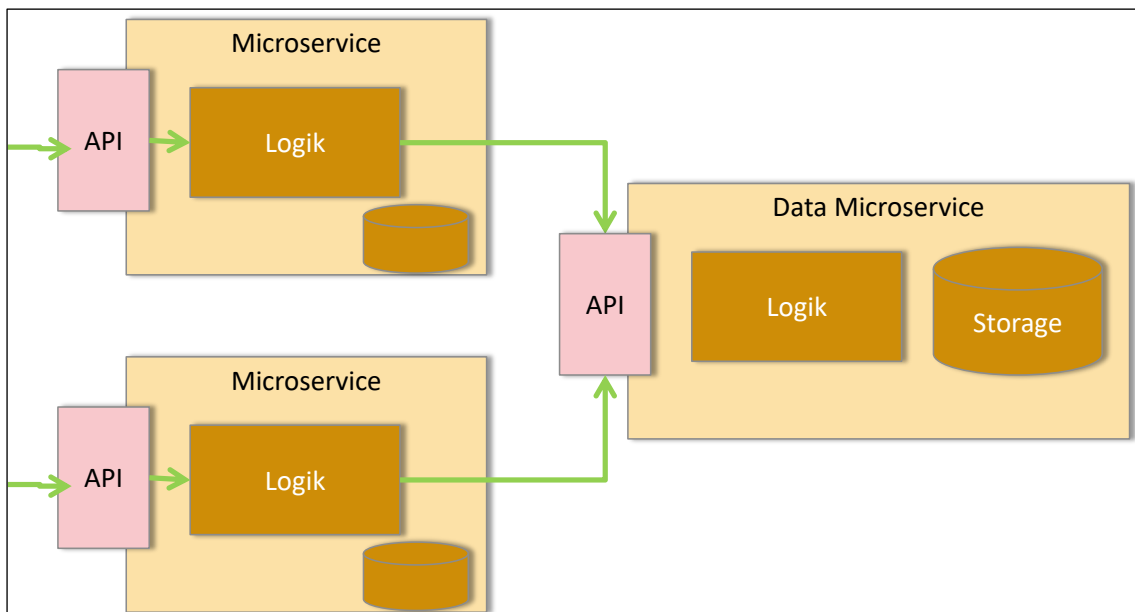
1.2.12 Synchronisation durch Implementierung



1.2.13 Synchronisation durch Data Pump



1.2.14 Synchronisation durch Data Service



1.3 Modellierung

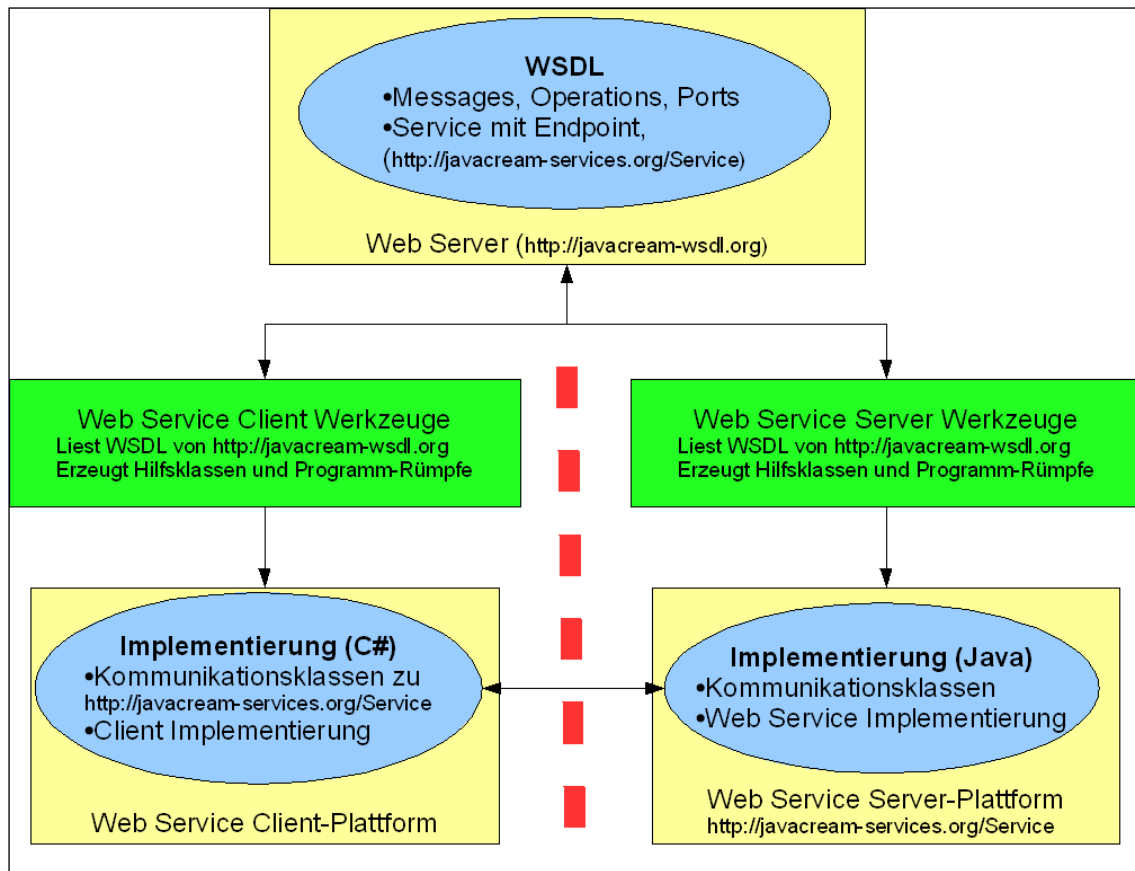
1.3.1 Web-Services mit SOAP – Eine Lösung?

- SOAP-basierte Web-Services verlangen eine Schnittstellen-Beschreibung
 - Formulierung in der Web Services Description Language, WSDL
 - WSDL ist ein XML-Schema zur Beschreibung eines Interfaces bestehend aus
 - Typen
 - Operationen
 - Ports
 - Eine WSDL ist damit ein XML-Dokument
- Ein SOAP-Envelope definiert den Aufruf
 - SOAP ist ebenfalls ein XML-Schema
 - Das SOAP-Dokument wird an die Zieladresse des Web-Services, den Endpoint gesendet

1.3.2 Benutzung von SOAP-basierten Web-Services

- Definiert wird eine Service-Beschreibung in Form einer WSDL
- Realisierung
 - Für die Server-Implementierung werden Code-Rümpfe erzeugt,
 - für den Client fertige Stub-Klassen
- Beim Aufruf werden die SOAP-Envelopes über das Netzwerk ausgetauscht

1.3.3 Beispiel eines SOAP-Web Services



1.3.4 Das Web interessiert sich nicht für SOAP!

- Keine einzige Browser-Implementierung erzeugt beispielsweise beim Formular-Submit einen SOAP-Envelope
- JavaScript könnte dies zwar prinzipiell, es gibt aber hierfür keine einzige ernst zu nehmende Utilities-Bibliothek
- Hardware und Software von Systemen sind auf http-Requests ausgerichtet
 - Firewalls
 - Load Balancer und Router
 - Cache-Lösungen

1.3.5 Der Representational State Transfer

- Doktorarbeit von Roy Fielding, 2000
- Siehe <http://en.wikipedia.org/wiki/REST>

Representational state transfer

From Wikipedia, the free encyclopedia
(Redirected from [REST](#))

"REST" redirects here. For other uses, see [Rest](#).

Representational state transfer (REST) is a software architectural style consisting of a coordinated set of architectural constraints applied to components, connectors, and data elements, within a distributed [hypermedia](#) system. REST ignores the details of component implementation and protocol syntax in order to focus on the roles of components, the constraints upon their interaction with other components, and their interpretation of significant data elements.^{[1][2]}

The term *representational state transfer* was introduced and defined in 2000 by [Roy Fielding](#) in his doctoral dissertation at [UC Irvine](#).^{[1][3]} REST has been applied to describe desired web architecture, to identify existing problems, to compare alternative solutions, and to ensure that protocol extensions would not violate the core constraints that make the web successful. Fielding used REST to design [HTTP 1.1](#) and [Uniform Resource Identifiers \(URI\)](#).^{[4][5]} The REST architectural style is also applied to the development of [web services](#)^[6] as an alternative to other distributed-computing specifications such as [SOAP](#).

Contents [\[hide\]](#)

- [1 History](#)
- [2 Software architecture](#)
 - [2.1 Components](#)
 - [2.2 Connectors](#)
 - [2.3 Data](#)
- [3 Architectural properties](#)
- [4 Architectural constraints](#)
 - [4.1 Client-server](#)
 - [4.2 Stateless](#)
 - [4.3 Cacheable](#)
 - [4.4 Layered system](#)
 - [4.5 Code on demand \(optional\)](#)
 - [4.6 Uniform interface](#)

1.3.6 Kern-Konzepte

- Die Arbeitsweise des Internets abstrahiert
 - Was sind Ressourcen?
 - Wie werden Ressourcen identifiziert?
 - Was sind Ressourcen-Operationen?
 - Wie werden Services beschrieben?
 - Was sind Stateless Operationen?
 - Voraussetzungen und Umsetzung von Caching-Mechanismen
 - Optional: Übertragung von Skript-Logik auf den Client
- Grundlegendes Konzept sind die verlinkten HyperText-Dokumente
- Nicht überraschend: „Das Internet ist ein Beispiel für die Implementierung eines REST-basierten Systems“

1.3.7 Das http-Protokoll

- Eine umfassende Spezifikation des w3w-Konsortiums
- Siehe <http://en.wikipedia.org/wiki/Http>

The screenshot shows the Wikipedia article for 'Hypertext Transfer Protocol'. The title is 'Hypertext Transfer Protocol'. Below the title, it says 'From Wikipedia, the free encyclopedia' and '(Redirected from Http)'. The main text describes HTTP as an application protocol for distributed, collaborative, hypermedia information systems. It mentions that HTTP is the foundation of data communication for the World Wide Web. A sidebar on the left contains a 'Contents' list with links to various sections like 'Technical overview', 'History', 'HTTP session', 'Request methods', 'Status codes', 'Persistent connections', 'HTTP session state', 'Encrypted connections', 'Request message', 'Response message', and 'Example session'. A sidebar on the right titled 'Internet protocol suite' lists protocols across four layers: Application layer (BGP, DHCP, DNS, FTP, HTTP, IMAP, IRC, LDAP, MGCP, NNTP, NTP, POP, RPC, RTP, RTSP, RIP, SIP, SMTP, SNMP, SOCKS, SSH, Telnet, TLS/SSL, XMPP), Transport layer (TCP, UDP, DCCP, SCTP, RSVP), Internet layer (IP, ICMP, ICMPv6, ECN, IGMP, IPsec), and Link layer (ARP, InARP, NDP, OSPF, Tunnels, PPP, Media access control, DSL, ISDN, FDDI, DOCSIS).

1.3.8 Elemente der http-Spezifikation

- Definition von URIs
 - Pfad
 - Parameter
- http-Request und http-Response
 - Daten-Container mit Header und Body
 - Encodierung
- Umfassender Satz von Header-Properties
 - Content-Length
 - Accepts
 - Content-Type

1.3.9 Elemente der http-Spezifikation II

- http-Methoden
 - PUT
 - GET
 - POST
 - DELETE
 - OPTIONS
 - HEAD
- Statuscodes für Aufrufe
 - 404: „Not found“
 - 204: „Created“
 - ...

1.3.10 MimeTypes

- Definition der Datentypen des Internets
 - Nicht zu verwechseln mit einem XML-Schema
 - Ein MimeType ist „nur“ eine strukturierte Zeichenkette
 - Eigene Erweiterungen sind möglich

1.3.11 REST und http

- REST hat mit http prinzipiell nichts zu tun
 - REST ist eine abstrakte Architektur
 - http ist ein konkretes Kommunikationsprotokoll
- Aber
 - http passt als Kommunikations-Protokoll der „Referenz-Implementierung“ Internet natürlich perfekt zum REST-Stil

1.3.12 Mapping REST – http

- http Methoden und Ressourcen-Operationen
 - PUT
 - Neu-Anlegen einer Ressource
 - Aktualisierung
 - GET
 - Lesen einer Ressource

- POST
 - Aktualisierung
 - Neuanlage
- DELETE
 - Löschen
- Idempotenz
 - Idempotente Operationen dürfen beliebig oft aufgerufen werden und verursachen keine Nebeneffekte
 - REST verlangt eine idempotente Implementierung für PUT und DELETE

1.3.13 Konzeption eines RESTful Services: Neuanlage

- Mit PUT
 - Der Client muss die Ressourcen-ID mit angeben
 - Rückgabe ist ein Statuscode „201: Created“
- Mit POST
 - Der Server entscheidet, ob er eine neue Ressource anlegen muss
 - Falls ja:
 - Statuscode „201: Created“
 - Gesetzter `Location`-Header mit URI der eben angelegten Ressource
 - Optional: Body enthält die angelegte Ressource

1.3.14 Konzeption eines RESTful Services: Update

- Mit PUT
 - Statuscode „200: OK“ oder „204: No content“
 - PUT ist idempotent (!)
- Mit POST
 - POST wird für nicht-idempotente Updates benutzt

1.3.15 Konzeption eines RESTful Services: Delete

- Mit DELETE
 - Statuscode „200: OK“ oder „204: No content“
 - PUT ist idempotent (!)
- Konzeptionell muss unterschieden werden:
 - Ein „echtes“ DELETE löscht die Ressource
 - Ein fachliches Löschen (z.B. Storno) ist eigentlich ein Update der Ressource
 - Ein Überladen des http-DELETE ist für diese Zwecke jedoch durchaus legitim
 - DELETE order/ISBN42?cancel=true

1.3.16 Service-Beschreibung

- Ein RESTful Web Service benötigt in erster Näherung keine gesonderte Beschreibung
 - PUT, GET, POST, DELETE decken alle relevanten Ressourcen-Operationen ab
 - Datentypen werden durch MIME-Types bestimmt
- Die einzige notwendige Information ist die Endpoint-Adresse
- In der Praxis werden jedoch in den meisten Fällen zusätzliche Informationen benötigt
 - Ein GET auf den Endpoint liefert beispielsweise eine Liste aller Ressourcen
- Aber:
 - Liste aller Informationen oder Referenzen?
 - Bestimmung der Sortierreihenfolge?
 - ...

1.3.17 REST Services

- Analog zur WSDL wird eine detailliertere Service-Beschreibung geliefert
- Web Application Description Language (WADL)
 - http://en.wikipedia.org/wiki/Web_Application_Description_Language
- Rest Service Description Language (RSDL)
 - <http://en.wikipedia.org/wiki/RSDL>
- Swagger
 - <http://swagger.io>

1.3.18 Ein komplett anderer Ansatz: HATEOAS

- Hypertext as the Engine of Application State
 - <http://en.wikipedia.org/wiki/HATEOAS>
- Grundidee: Jeder Response enthält eine Menge von Verlinkungen, die die weiteren sinnvollen Aktionen definieren
 - „Hört sich an, wie eine Web Anwendung!“
 - Aber nun konsequent mit PUT, DELETE

1.3.19 Wie groß ist "Micro"

- In der Literatur zu finden: "So klein wie möglich"
 - So natürlich unsinnig
 - Der kleinstmögliche Microservice
 - enthält exakt eine Operation
 - liefert ein primitives Ergebnis
- Besser: "So groß wie nötig"
 - Kriterien:
 - Aufwand der Daten-Replikation über verschiedene Microservices hinweg
 - Transaktions-Kontexte
 - Performance-Aspekte

2

Software-Entwicklung

2.1	Java, JEE und Spring Boot.....	2-3
2.1.1	Java	2-3
2.1.2	JEE	2-3
2.1.3	Spring Source	2-3
2.1.4	Spring Boot	2-4
2.2	Build-Prozess	2-5
2.2.1	Kriterien für den Build-Prozess	2-5
2.2.2	Bestandteile	2-5
2.2.3	Beispiel: Apache Maven	2-5
2.2.4	Build-Prozess für Java-Projekte.....	2-6
2.2.5	Apache Maven.....	2-6
2.2.6	Maven-Konfiguration: Das interne Repository	2-7
2.2.7	Maven-Konfiguration: Ausbringen eigener Artefakte.....	2-7
2.2.8	Weitere Elemente der Parent-POM	2-8
2.2.9	Beispiel: Sonatype Nexus	2-8
2.2.10	Grundsätzliche Organisation.....	2-9
2.2.11	Konfiguration.....	2-9
2.2.12	Nexus Maven-Repository.....	2-10
2.3	Integration und Überwachung.....	2-11
2.3.1	Problemstellung	2-11
2.3.2	Jolokia.....	2-11

2 Software-Entwicklung

2.1 Java, JEE und Spring Boot

2.1.1 Java

- Die Java Virtual Machine ist eine gekapselte Laufzeitumgebung für JAvA-Anwendungen
- Parallele Ausführung auf einer Hardware problemlos möglich
- Anbindung an System-Überwachung ist gegeben

2.1.2 JEE

- Die Java Enterprise Edition (JEE) erweitert die Java Virtual Machine um weitere Infrastruktur
 - Applikationsserver
- Der Applikationsserver führt JEE-Anwendungen aus
 - Kapselung innerhalb des Applikationsservers
 - Hot Deployment
 - Inter-App-Kommunikation
 - In Memory durch Verlinkung
 - Internes Bus-Systems
 - Insbesondere Messaging
 - Unterstützung aller geläufiger Kommunikationsprotokolle

2.1.3 Spring Source

- Ein weit verbreitetes Open Source-Framework als Alternative zum JEE-Applikationsserver
- Spring-Core-Komponente und viele assoziierte Projekte
 - Spring Data
 - Spring MVC
 - Spring Integration
 - ...

2.1.4 Spring Boot

- Eine Spring-Boot-Applikation wird als ein einziges Deployment-Archiv ausgeliefert
und über einen simplen Java-Aufruf gestartet
- Damit passt Spring Boot hervorragend zu Docker
 - Als Grundlage dient ein simples Java Image
 - Die zu installierende Applikation wird als eine einzige Datei hinzugefügt
 - und über einen Standard-Java-Aufruf gestartet

2.2 Build-Prozess

2.2.1 Kriterien für den Build-Prozess

- Reproduzierbar
- Plattform-übergreifend
 - Developer-Rechner
 - Build-Machine
- Erzeugt "Artefakte"

2.2.2 Bestandteile

- Source Code Management
 - Enthält die Quellcodes, Konfigurationen der Anwendung
 - die so genannten "Werke"
 - Auch die Build-Prozesse sind Werke
- Build-Maschine
 - Enthält die Build-Jobs und führt diese aus
- Artefakt-Repository
 - Die vom Build-Prozess erzeugten Ergebnisse werden Artefakte genannt
 - Diese werden in versionierter Struktur in einem Repository gehalten

2.2.3 Beispiel: Apache Maven

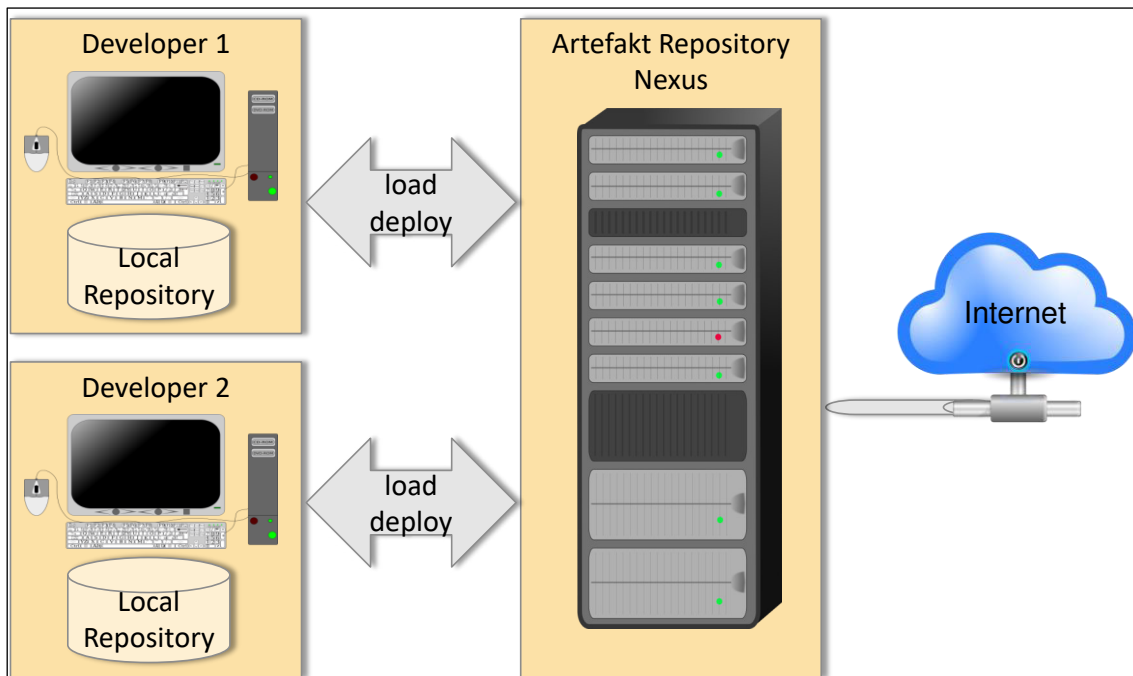
- In der Java-basierten Software-Entwicklung weit verbreitet
- Bestandteile
 - Fester Build-Prozess
 - Aufruf mit mvn <command>
 - compile
 - package
 - install
 - deploy
- Lokaler Cache für gebildete Artefakte

- Connectivity zu einem Remote-Repository
 - Standard: Internet-Repository repo1.maven.org
 - Im Unternehmen: Eigener Repository-Server
 - Nexus
 - Artefactory
 - ...

2.2.4 Build-Prozess für Java-Projekte

- Build-Sequenz
 - Compiler-Aufruf
 - JAR-Archivierer
 - + x, z. B. Auschecken aus Versionsverwaltung, Testen, Ausbringen aus Servern, etc.
- Dependency Management
 - Andere interne Software-Projekte
 - Open Source-Projekte
 - Produkt-Bibliotheken

2.2.5 Apache Maven



2.2.6 Maven-Konfiguration: Das interne Repository

- settings.xml, allgemein gültig

```
<settings>
  <mirrors>
    <mirror>
      Hier wird die Adresse des internen Servers eingetragen
    </mirror>
  </mirrors>
  <servers>
    <server>
      Authentifizierungs-Informationen für den internen Server
    </server>
  </servers>
</settings>
```

2.2.7 Maven-Konfiguration: Ausbringen eigener Artefakte

- Das übernimmt eine so genannte Parent-POM
 - Eine Art Superklasse für alle Maven-Buildprozesse
 - Diese wird als eigenes Artefakt im Repository abgelegt
- Beispiel:
 - Ausschnitt des Distribution-Managements

```
<distributionManagement>
  <repository>
    <uniqueVersion>false</uniqueVersion>
    <id>nexus</id>
    <name>Corporate Repository</name>
    <url>http://...</url>
  </repository>
  <snapshotRepository>
    <uniqueVersion>true</uniqueVersion>
    <id>nexus</id>
    <name>Corporate Snapshots</name>
    <url>http://...</url>
  </snapshotRepository>
</distributionManagement>
```

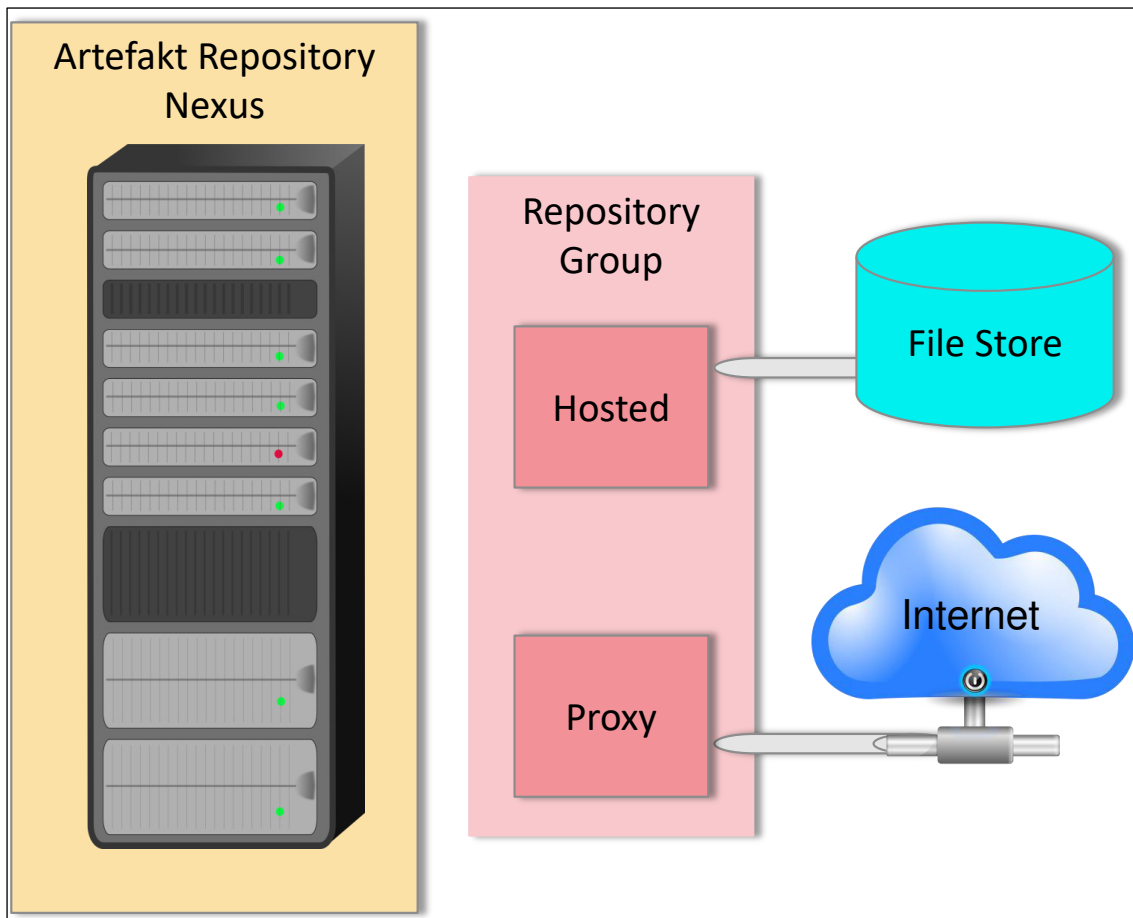
2.2.8 Weitere Elemente der Parent-POM

- Konfiguration des Java-Compilers
 - Java-Version
- Reporting
- Allgemein zu benutzende Dependencies zu anderen Artefakten
 - JUnit
 - Datenbank-Treiber
 - ...
- Konfiguration weiterer PlugIns
 - Spezielle Reports

2.2.9 Beispiel: Sonatype Nexus

- Fertige Produktlösung von Sonatype
 - Community
 - Kommerzieller Support
- Repository Server für
 - Java Artefakte
 - Aber auch Docker-Images
- Alternative Produkte
 - Apache Archiva
 - JFrog Artefactory
 - Atlassian Bitbucket

2.2.10 Grundsätzliche Organisation



2.2.11 Konfiguration

- Die Repository -Group wird für Lese-Vorgänge genutzt
 - Nicht vorhandene Artefakte oder Images werden über den Internet-Proxy nachgeladen
- Das Hosted-Repository wird für die Ablage eigener Artefakte und Images genutzt

2.2.12 Nexus Maven-Repository

The screenshot shows the Nexus Repository Manager web interface in a Mozilla Firefox browser. The address bar displays `localhost:8081/#browse/search/maven`. The page title is "Maven - Nexus Repository Manager". The left sidebar shows the "Browse" menu with options like "Welcome", "Search", "Custom", "Docker", "Maven", "NuGet", "Assets", and "Components". The "Maven" option is selected. The main content area shows the "Maven" search results for components by Maven coordinates. The search filters are set to "Any" for Group Id, Artifact Id, Version, Base Version, Classifier, and Extension. The results table shows the first 1,000 of 1,145 results. The table has columns for Name, Group, Version, and Format. The results list various artifacts from the `org.javacream.training.application.book` group, including `book`, `book.api`, `book.cdi`, `book.cdi.jee`, `book.cdi.jee.jpa`, `book.cdi.jee.memory`, and `book.cdi.spring`.

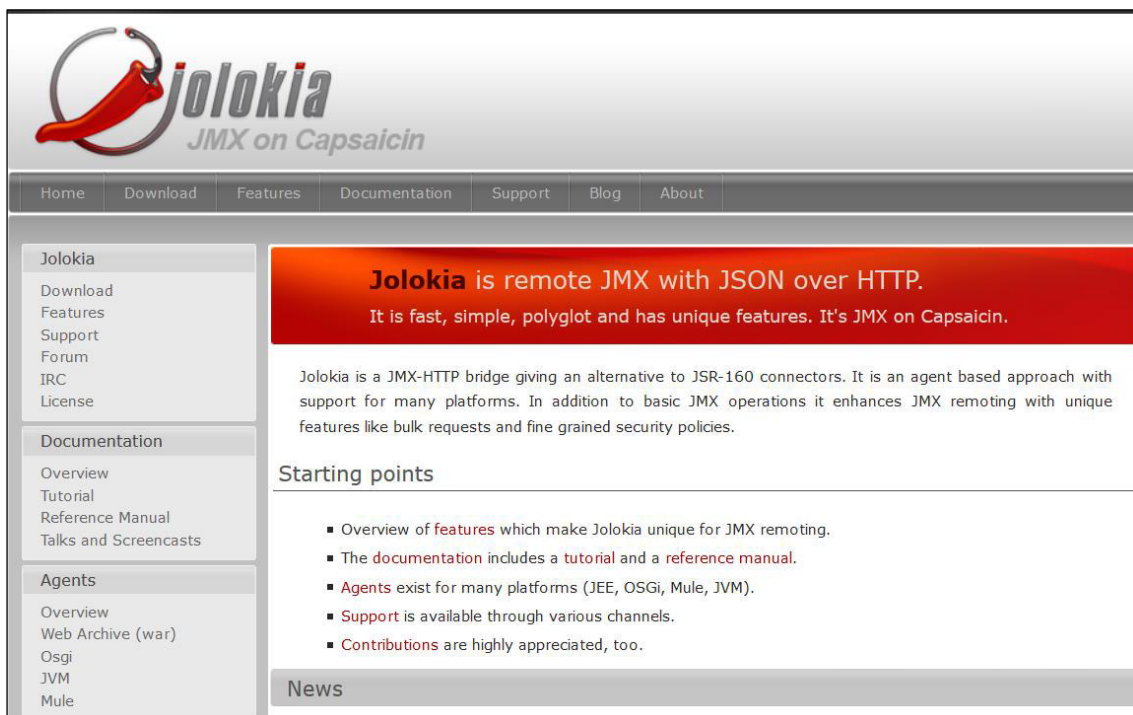
Name	Group	Version	Format
org.eclipse.sisu.plexus	org.eclipse.sisu	0.3.0	maven2
org.javacream.training.application.book	org.javacream.training.application.book	0.0.1-20170310...	maven2
org.javacream.training.application.book	org.javacream.training.application.book	0.0.1-20170310...	maven2
org.javacream.training.application.book.api	org.javacream.training.application.book	0.0.1-20170305...	maven2
org.javacream.training.application.book.api	org.javacream.training.application.book	0.0.1-20170310...	maven2
org.javacream.training.application.book.api	org.javacream.training.application.book	0.0.1-20170310...	maven2
org.javacream.training.application.book.cdi	org.javacream.training.application.book.cdi	0.0.1-20170310...	maven2
org.javacream.training.application.book.cdi	org.javacream.training.application.book.cdi	0.0.1-20170310...	maven2
org.javacream.training.application.book.cdi.jee	org.javacream.training.application.book.cdi	0.0.1-20170310...	maven2
org.javacream.training.application.book.cdi.jee	org.javacream.training.application.book.cdi	0.0.1-20170310...	maven2
org.javacream.training.application.book.cdi.jee.jpa	org.javacream.training.application.book.cdi	0.0.1-20170310...	maven2
org.javacream.training.application.book.cdi.jee.jpa	org.javacream.training.application.book.cdi	0.0.1-20170310...	maven2
org.javacream.training.application.book.cdi.jee.memory	org.javacream.training.application.book.cdi	0.0.1-20170310...	maven2
org.javacream.training.application.book.cdi.jee.memory	org.javacream.training.application.book.cdi	0.0.1-20170310...	maven2
org.javacream.training.application.book.cdi.spring	org.javacream.training.application.book.cdi	0.0.1-20170310...	maven2
org.javacream.training.application.book.cdi.spring	org.javacream.training.application.book.cdi	0.0.1-20170310...	maven2

2.3 Integration und Überwachung

2.3.1 Problemstellung

- Die Betriebssystem-nahe Überwachung ist für Java-Prozesse nicht sonderlich aufschlussreich
 - Metriken der Java-Virtual Machine
 - Heap-Speicher
 - Garbage Collections
- Hierfür wird besser JMX benutzt
 - Idee: Auf einem (im Unternehmen standardisierten) Port wird JMX beispielsweise mit Jolokia bereitgestellt
 - Hierfür existieren aber auch andere Lösungen

2.3.2 Jolokia



3

Container für Microservices

3.1	Applikationsserver	3-3
3.1.1	Allgemeines	3-3
3.1.2	Deployment.....	3-3
3.1.3	Überwachung.....	3-3
3.1.4	Der Applikationsserver.....	3-4
3.1.5	Inbound Connectors.....	3-4
3.1.6	Aufgaben des Inbound Connectors	3-5
3.1.7	JEE Komponenten.....	3-5
3.1.8	JEE Komponenten.....	3-5
3.1.9	Outbund Connectors.....	3-6
3.1.10	Beispiele für Outbound Connectors	3-6
3.2	Docker	3-7
3.2.1	Was ist ein Container?.....	3-7
3.2.2	Container-Isolation.....	3-7
3.2.3	Bestandteile	3-7
3.2.4	Docker und (Micro)Services.....	3-8
3.2.5	Microservices mit Docker-Containern	3-8
3.2.6	Starten eines ersten Containers	3-9
3.2.7	Docker-Workflow: 1. und 2. Lauf.....	3-10
3.2.8	Lebenszyklus des Containers	3-10
3.2.9	Docker im Gesamtbild.....	3-10

3.2.10	Images und Layers	3-11
3.2.11	Beispiel: Ein Java-Image (https://microbadger.com/images/java)	3-11
3.2.12	Container	3-12
3.2.13	Inter-Container-Kommunikation	3-12
3.2.14	Kommandoreferenz	3-13
3.2.15	Workflow	3-13
3.2.16	Dockerfile für eine Java-Umgebung.....	3-14
3.2.17	Dockerfile Referenz	3-14
3.2.18	Gekapselte Services mit Docker.....	3-15

3 Container für Microservices

3.1 Applikationsserver

3.1.1 Allgemeines

- Applikationsserver sind Bestandteil der Java Enterprise Edition
 - Eine Spezifikation
- Ausprägungen:
 - Web Profile
 - Web Anwendungen
 - Web Services
 - Full Profile
 - Transaktionelle Ressourcen-Zugriffe
 - Messaging
 - Weitere inoffizielle Profile
 - Reine Messaging Systeme
 - Web Profile mit einfachem Transaktionsmanagement

3.1.2 Deployment

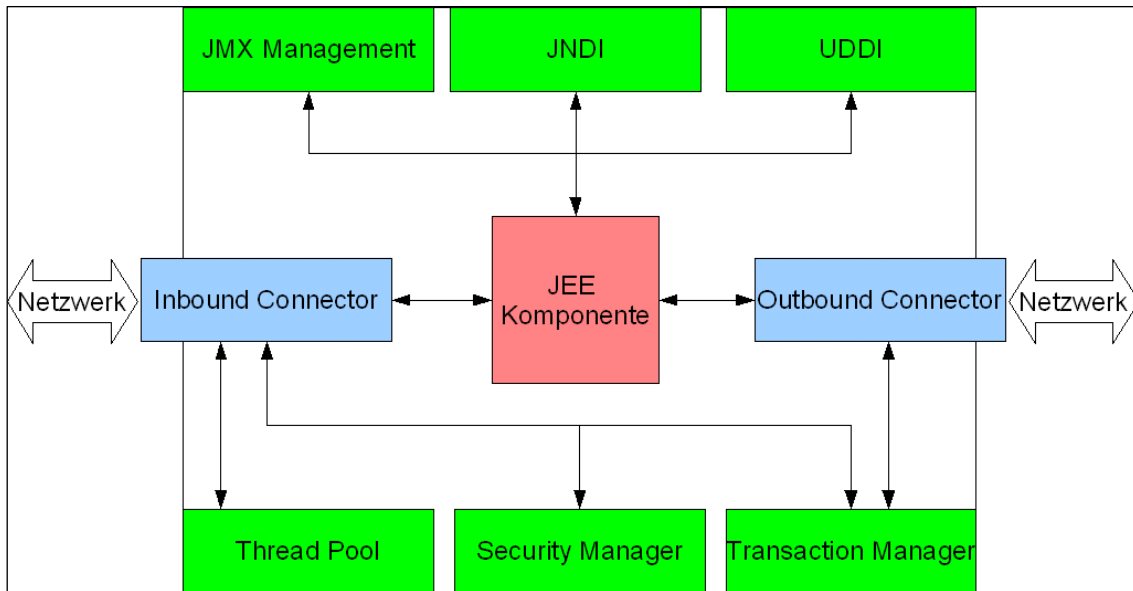
- Ausbringen von Anwendungen in den laufenden Server
 - Hot Deployment
 - File-Transfer in ein überwachtes Verzeichnis
 - Administrativer Vorgang
 - Web Console
 - Admin-Skripte
 - Konfigurativ
 - mit Neustart des Servers

3.1.3 Überwachung

- Der Server ist über JMX zugreifbar
- Web Konsolen
- Log-Dateien
- Betriebssystem-Überwachung
 - Speicher
 - CPU

- IO
- Bei Verwendung des Docker-Containers übernimmt dieser die Rolle des Betriebssystems!

3.1.4 Der Applikationsserver



3.1.5 Inbound Connectors

- Inbound Connectors ermöglichen die Kommunikation mit dem Applikationsserver
 - http/https
 - Java Remote Method Invocation (RMI) bzw. die interoperable Variante RMI/IIOP
 - Empfangen von Messages mit dem Java Messaging Service (JMS)
 - SOAP
- Für Protokolle, die nicht im Standard enthalten sind, können spezielle Implementierungen für Inbound Connectors als Plug-In in den Applikationsserver installiert werden
 - Dafür existiert auch ein breiter Markt von Herstellern und Produkten

3.1.6 Aufgaben des Inbound Connectors

- Die Inbound Connectors enthalten selber keine Geschäftsprozesse
- Laut Konzeption wandeln sie eine einkommende Netzwerkverbindung um in einen Request der, wie oben bereits angesprochen, von einem Thread aus dem Thread Pool ausgeführt wird
- Geschäftslogik wird in einer JEE-Komponente definiert
- Ein Inbound Connector kann im Rahmen einer Verteilten Transaktion aufgerufen werden

3.1.7 JEE Komponenten

- JEE Komponenten bestehen aus zwei Komponenten:
 - Einem Container, der vom Applikationsserver zur Verfügung gestellt wird
 - und der eigentlichen Komponente, die als Bestandteil eines Anwendungsprogramms aufzufassen ist

3.1.8 JEE Komponenten

- Servlets werden vom http Inbound Connector aufgerufen
 - Die Entscheidung, welches Servlet (und damit welche Anwendungslogik) aufgerufen werden soll erfolgt über die URL des http-Aufrufs
 - Ein spezieller Typ von Servlet kann auch über SOAP aufgerufen werden
 - In modernen Anwendungen existiert wahrscheinlich nur ein einziges zentrales Servlet
- Enterprise JavaBeans dienen primär zur Transaktionssteuerung
 - Dem Anwendungsprogrammierer werden verschiedene Sub-Typen angeboten, die sich bezüglich Lebenszyklus und Aufruf-Protokoll unterscheiden
 - Es gibt „Stateless“ und „Stateful SessionBeans“, „Message Driven Beans“ sowie „Singletons“
 - Enterprise JavaBeans sind in modernen Anwendungen häufig nicht mehr anzutreffen

3.1.9 Outbund Connectors

- Outbound Connectors werden von JEE-Komponenten lokal benutzt
- Ihre Aufgabe ist es, einen Geschäftsprozess innerhalb eines Backend-Systems aufzurufen
 - Dazu hält der Outbound Connector einen Connection Pool
- Die Outbound Connectors werden auch vom Transaktionsmanager (für die Koordinierung der Verteilten Transaktionen) und vom Security Manager (Propagierung des angemeldeten Benutzers) benutzt

3.1.10 Beispiele für Outbound Connectors

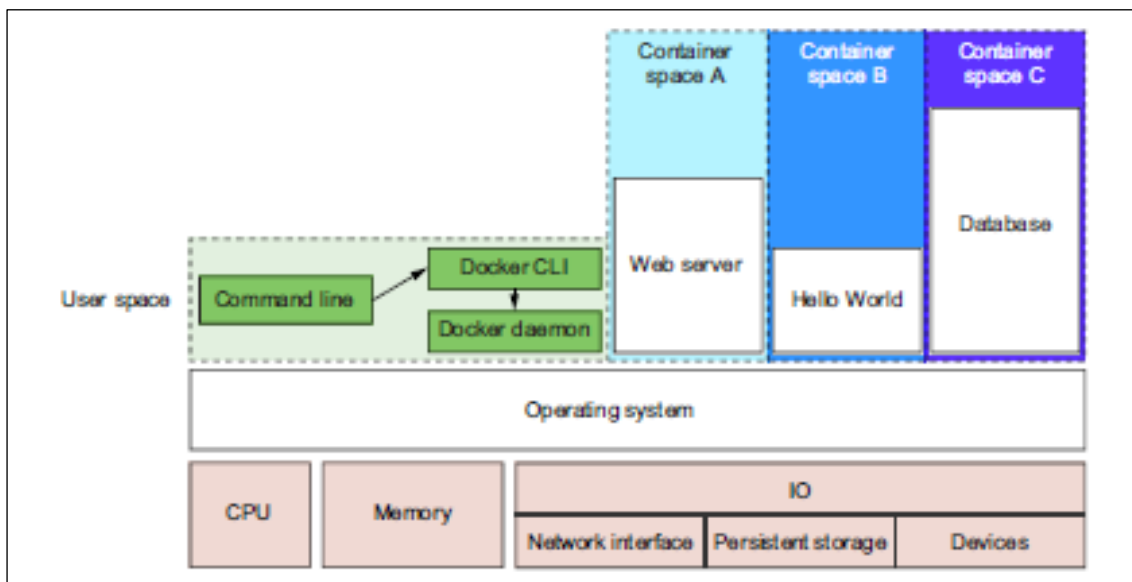
- Die JEE-Spezifikation verlangt die folgenden Outbound Connectors:
- Die Data Sources verbinden zu SQL-Datenbanksystemen
- Versenden von JMS Messages
- Anbinden an ein E-Mail-System
- Alle weiteren Systeme sind wiederum durch Plug-Ins installierbar, für die ein breiter Markt existiert

3.2 Docker

3.2.1 Was ist ein Container?

- "Ein Container ist eine (modifizierte) Laufzeitumgebung, die der darin laufenden Anwendung den Zugriff auf geschützte Ressourcen unmöglich macht."
 - Unterschied zu Virtualisierung: Hier wird eine komplette Hardware durch eine "Virtuelle Maschine" abstrahiert
- Docker-Container laufen direkt ohne weitere Emulation auf dem Linux-Kernel
 - dabei werden etablierte Linux-Features wie namespaces und cgroups benutzt

3.2.2 Container-Isolation



3.2.3 Bestandteile

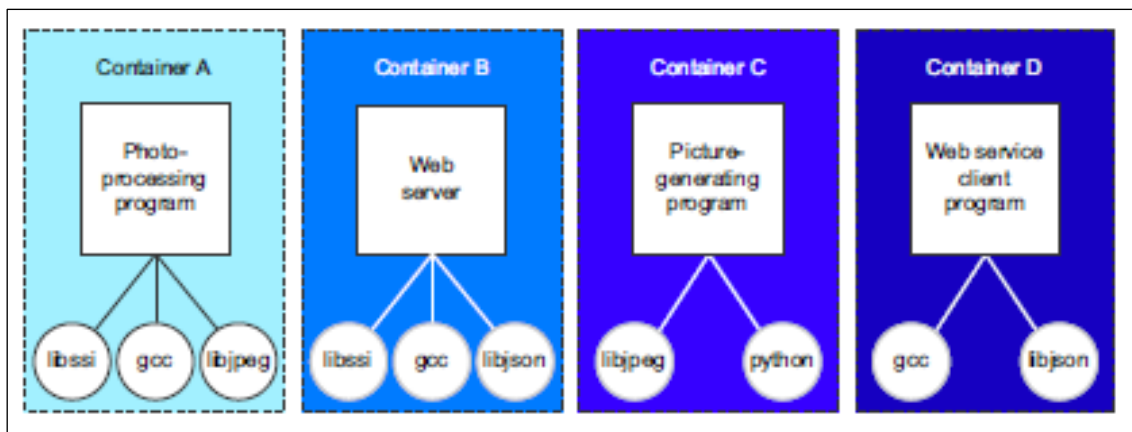
- Docker Command Line
 - Dieser Befehl wird vom Docker-Anwender direkt benutzt
- Docker Host
 - Eine Maschine mit installierter Docker Umgebung
 - Docker Dämon/Engine
 - Lokale Registry

- Docker Dämon/Engine
 - Über den Dämon werden die Docker-Container gestartet
 - Technisch gesprochen sind die Container Child-Prozesse des Dämons
- Remote Services
 - Registries für Docker Images
 - DockerHub

3.2.4 Docker und (Micro)Services

- Microservices
 - Wohl-definiertes API
 - hier nicht relevant
 - Netzwerk
 - Isolation der internen Implementierung
 - Bereitstellung aller benötigten Ressourcen
 - Interne Ressourcen wie Datenbanken
 - Dependencies auf andere Services
- Docker passt!

3.2.5 Microservices mit Docker-Containern



3.2.6 Starten eines ersten Containers

- Nach der Installation kann der erste Container gestartet werden (!)
 - `docker run <docker_image>`
z.B. `docker run hello-world`
- Das wars

Ausgabe

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
535020c3e8ad: Pull complete
af340544ed62: Pull complete
Digest:
sha256:a68868bfe696c00866942e8f5ca39e3e31b79c1e50feaaa4ce5e28df2f051d5c
Status: Downloaded newer image for hello-world:latest
Hello from Docker.

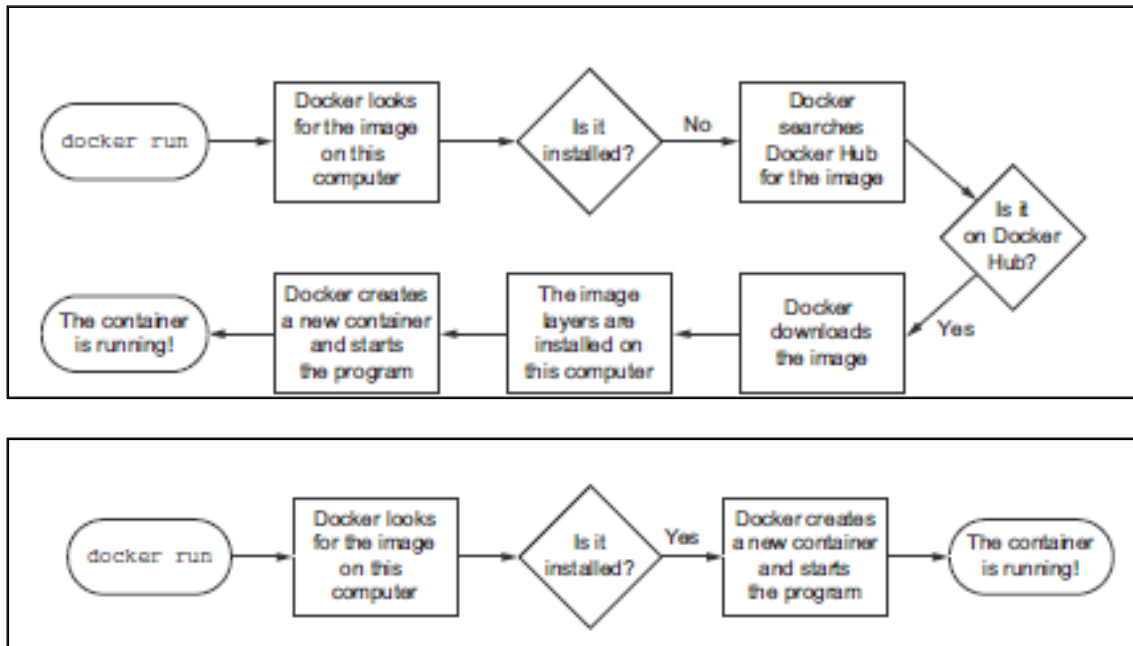
This message shows that your installation appears to be working correctly.
To generate this message, Docker took the following steps:
1. The Docker Engine CLI client contacted the Docker Engine daemon.
2. The Docker Engine daemon pulled the "hello-world" image from
   the Docker Hub.
3. The Docker Engine daemon created a new container from that image which
   runs the executable that produces the output you are currently reading.
4. The Docker Engine daemon streamed that output to the Docker Engine
   CLI client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker Hub account:
https://hub.docker.com

For more examples and ideas, visit:
https://docs.docker.com/userguide/
```

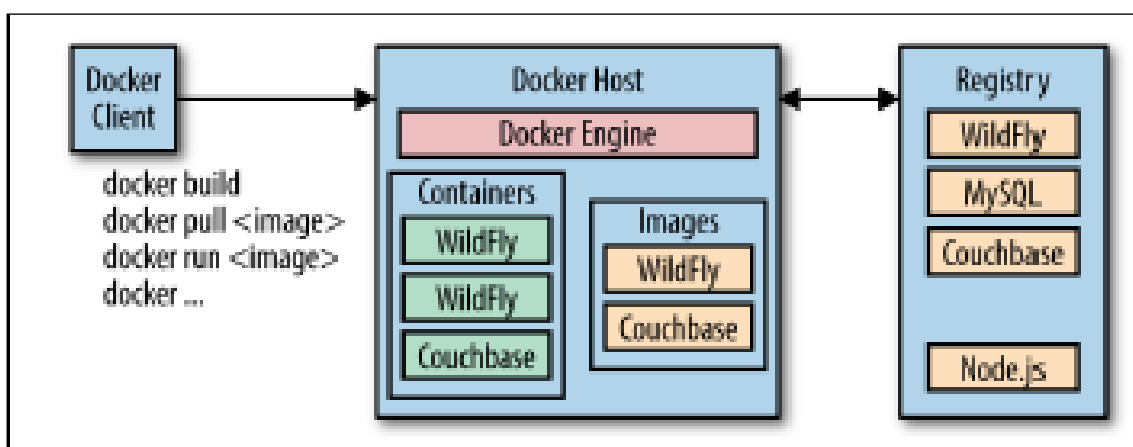
3.2.7 Docker-Workflow: 1. und 2. Lauf



3.2.8 Lebenszyklus des Containers

- Der Container wird gestartet
 - Nochmal: Keine VM, sondern ein gekapselter Prozess auf dem Host-System
- Innerhalb des Containers werden Prozesse gestartet
 - Diese werden sind der Definition des Images angegeben
- Der Container läuft, solange ein definierter Prozess läuft
 - Anschließend wird der Container selbst beendet

3.2.9 Docker im Gesamtbild



3.2.10 Images und Layers

- Stellt eine Read-Only-Umgebung zur Verfügung
 - Analogie OOP: Eine Klassen-Definition
- Im Gegensatz zu VMWare-Images aus Layern aufgebaut
 - Vererbung und Assoziation
- Durch die Layerung sind einzelne Images deshalb relativ schlank
- Images und Layers werden in Repositories verwaltet
 - Inklusive Meta-Informationen
 - Versionierung!
 - Die Docker-Client-Installation verwaltet ein lokales Repository

3.2.11 Beispiel: Ein Java-Image (<https://microbadger.com/images/java>)

The screenshot displays the Docker Hub interface for the `openjdk:8-jdk` image. At the top, there are tags for various versions: `openjdk-8u111`, `openjdk-8u111-jdk`, `openjdk-8`, `openjdk-8-jdk`, `latest` (selected), `jdk`, `8u111`, `8u111-jdk`, `8`, and `8-jdk`. Below the tags, the image details are listed: **Created** (January 17, 2017 at 01:52 AM), **ID** (d11c3799fa6a), **Download Size** (232.1 MB), and **Labels** (No labels). The **Layers** section shows 14 layers. The first layer, `buildpack-deps` (107.2 MB), is expanded to show its build process. The build process consists of several steps: adding a file, setting the command to `"/bin/bash"`, running `apt-get update` and `apt-get install -y --no-install-recommends b...`, running `apt-get update` and `apt-get install -y --no-install-recommends b...`, running `apt-get update` and `apt-get install -y --no-install-recommends b...`, running `echo 'deb http://deb.debian.org/debian jessie-backports main' > ...`, setting `ENV LANG=C.UTF-8`, running `{ echo '#!/bin/sh'; echo 'set -e'; echo; echo 'dirname "...`, setting `ENV JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64`, `ENV JAVA_VERSION=8u111`, `ENV JAVA_DEBIAN_VERSION=8u111-b14-2~bpo8+1`, setting `ENV CA_CERTIFICATES_JAVA_VERSION=20140324`, running `set -x && apt-get update && apt-get install -y openjdk-8-jdk...`, and finally running `/var/lib/dpkg/info/ca-certificates-java.postinst configure`.

3.2.12 Container

- Eine Instanz eines Images
 - Analogie zu OOP: Ein Objekt
 - Identifikation über einen Namen bzw. über einen technischen Schlüssel
 - Ein Hash
- Ein Container hat einen Lebenszyklus
 - Gesteuert über Docker-Kommandos
 - `create`
 - `run`
 - `start`
 - `stop`
 - `delete`
- Im Gegensatz zu Images können Container einen Zustand aufweisen
 - Container-Environment
 - Interne Ressourcen
 - Dateisystem des Containers

3.2.13 Inter-Container-Kommunikation

- Container sind komplett gekapselt
 - Interaktion nur über Docker-Kommandos
 - Auslesen des Docker-Logs
 - `Inspect`
 - Ausführen einer Shell im Container
- Benutzung externer Volumes
 - Ein Teil des Dateisystems des Containers vom Host zur Verfügung gestellt
- Netzwerk-Kommunikation
 - Container können Netzwerk-Sockets bereitstellen
 - Diese werden auf reale Sockets des Hosts gemapped
- Container-Linking
 - Direkte Kommunikation der Container untereinander
 - Verschiedene Abstufungen
 - Gemeinsames Netzwerk

- Gemeinsames File-Layer
- Shared Memory

3.2.14 Kommandoreferenz

- <https://docs.docker.com/engine/reference/commandline/docker/#/related-commands>
- Elementare Befehle
 - Container anlegen
 - `sudo docker create <<Image-Name>>:<<Versionsnummer>>`
 - Container löschen
 - `sudo docker rm <<containerId>>`
 - Docker-Container automatisch nach Ausführung löschen
 - `sudo docker run -rm <<Image-Name>>`
 - Alle Container anzeigen lassen
 - `sudo docker ps -a`
 - Container starten
 - `sudo docker start <<containerId>>`
 - Container anlegen, starten und assoziiert eigene Ausgabe-Konsole (nicht in der Terminal-Session, in der gearbeitet wird)
 - `sudo docker run --detach <<Image-Name>>`
 - Images anzeigen lassen
 - `sudo docker images`

3.2.15 Workflow

- Anlegen eines leeren Verzeichnisses
 - Alle Verzeichnisse und Unterverzeichnisse werden Bestandteil des Images
 - excludes in `.dockerignore`
- Erstellen eines Docker-Files
 - Bestehend aus Kommandos
 - FROM
 - COPY
 - ENV

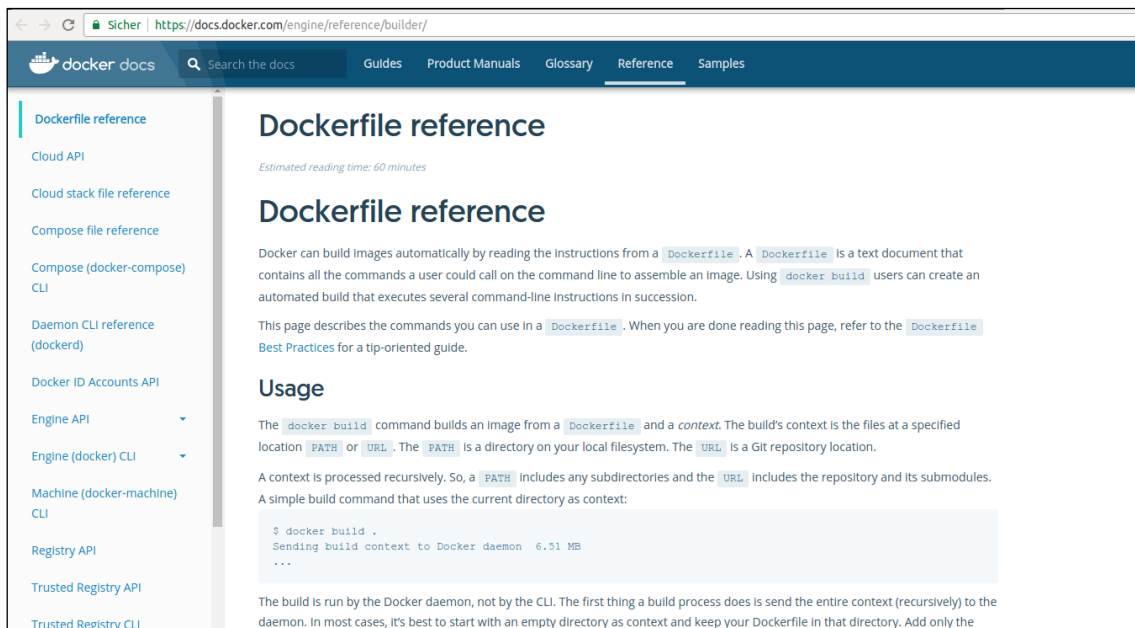
- RUN
- EXPOSE

3.2.16 Dockerfile für eine Java-Umgebung

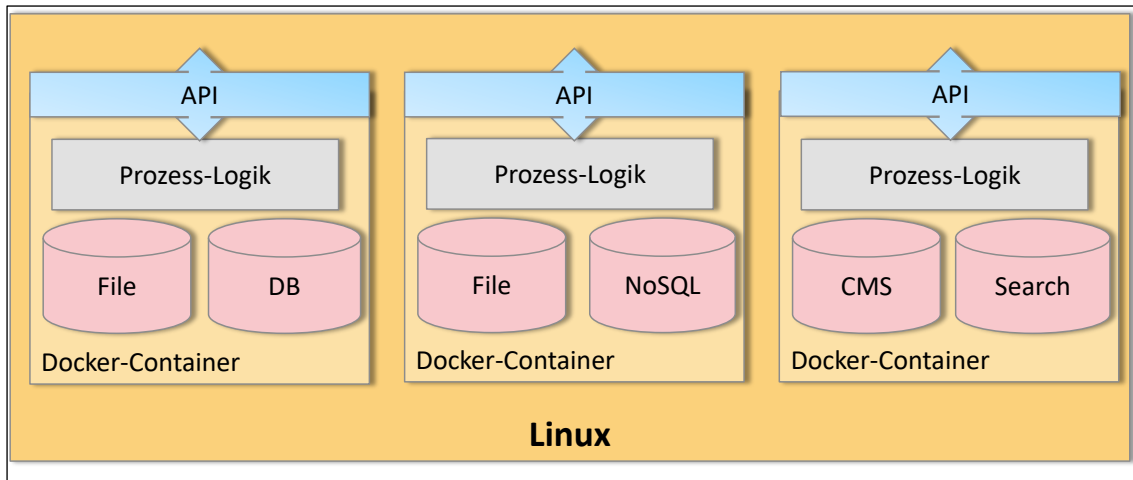
- Das Dockerfile selbst ist ein simples Text-Dokument
 - Referenz unter <https://docs.docker.com/engine/reference/builder/>
 - Beispiel

```
FROM openjdk
CMD ["java", "-version"]
```
- Erzeugen des Images
 - `docker build -tag <name>`

3.2.17 Dockerfile Referenz



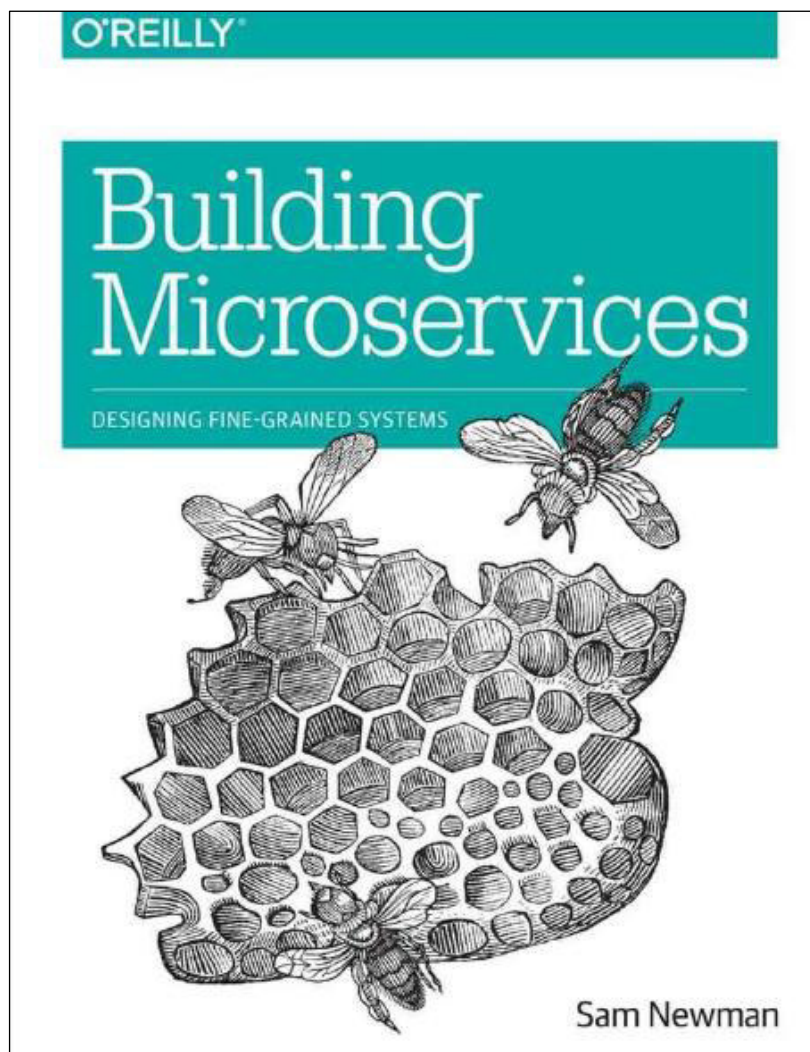
3.2.18 Gekapselte Services mit Docker



4

Literatur und Quellen

4 Literatur und Quellen





The screenshot shows the top portion of the Martin Fowler website. The header is dark with the text 'MARTINFOWLER.COM' in white. Below it is a navigation menu with links: 'Intro', 'Videos', 'Design', 'Agile', 'Refactoring', 'About Me', 'All Sections', and 'ThoughtWorks'. There are also icons for RSS and Twitter. The main content area has a white background with the title 'Microservices' in a large, bold, purple font. Below the title is a subtitle 'a definition of this new architectural term' in a smaller, bold, purple font. The main text is in an italicized serif font, describing the term 'Microservice Architecture'.

MARTINFOWLER.COM

Intro Videos Design Agile Refactoring About Me All Sections ThoughtWorks RSS Twitter

Microservices

a definition of this new architectural term

The term "Microservice Architecture" has sprung up over the last few years to describe a particular way of designing software applications as suites of independently deployable services. While there is no precise definition of this architectural style, there are certain common characteristics around organization around business capability, automated deployment, intelligence in the endpoints, and decentralized control of languages and data.